

Simulacro 3 – Técnicas de Diseño de Algoritmos (Segundo Parcial)

Duración sugerida: 3 horas

Total: 100 puntos

Ejercicio 1 – Multiple choice (10 puntos)

Para cada ítem elegir exactamente una opción. No es necesario justificar.

Cada respuesta correcta suma 2 puntos, incorrecta o con más de una opción marcada suma 0 puntos.

1. Sea un grafo no dirigido con n vértices y m aristas, representado con matriz de adyacencia.

¿Cuál es el costo asintótico de correr DFS desde un vértice origen s y alcanzar todo su componente conexo?

- a) $O(n + m)$
- b) $O(n^2)$
- c) $O(m \log n)$
- d) $O(m^2)$

2. En un grafo no dirigido y ponderado, se corre DFS y se clasifican las aristas en árbol, avance, retorno (back) y cruzadas.

¿Cuál de las siguientes afirmaciones es correcta?

- a) En un grafo no dirigido no existen aristas de retorno.
- b) En un grafo no dirigido no existen aristas de avance ni cruzadas (distintas de las de árbol).
- c) En un grafo no dirigido cualquier arista que no es de árbol es necesariamente de avance.
- d) En un grafo no dirigido cualquier arista que no es de árbol es necesariamente de retorno (back).

3. Sea un grafo no dirigido, conexo, con todos los pesos positivos y distintos.

¿Cuál de las siguientes afirmaciones es verdadera?

- a) Puede haber más de un árbol generador mínimo.
- b) El árbol generador mínimo es único.
- c) Prim y Kruskal pueden devolver árboles generadores mínimos diferentes.
- d) Si se suma la misma constante $k > 0$ a todas las aristas, el AGM puede cambiar.

4. En un grafo dirigido con pesos no negativos, se corre Dijkstra desde un origen s .

Luego se toma una arista (u, v) y se disminuye su peso (manteniendo todos los pesos ≥ 0).

¿Cuál de las siguientes afirmaciones es necesariamente correcta respecto a las distancias mínimas desde s ?

- a) Todas las distancias permanecen iguales.
- b) Todas las distancias disminuyen.
- c) Algunas distancias pueden disminuir, pero nunca aumentar.
- d) Algunas distancias pueden aumentar.

5. Sea una red de flujo con capacidades enteras no negativas. Se corre Ford–Fulkerson siempre eligiendo caminos aumentantes arbitrarios.

¿Cuál de las siguientes afirmaciones es correcta?

- a) El algoritmo puede no terminar nunca, incluso con capacidades enteras.
- b) El algoritmo siempre termina y encuentra un flujo máximo.
- c) El algoritmo siempre termina, pero puede no encontrar un flujo máximo.
- d) El algoritmo solo termina si se restringe a elegir caminos aumentantes de longitud mínima (Edmonds–Karp).

Tema que evalúa: representación vs complejidad de recorridos, clasificación de aristas en no dirigidos, unicidad del AGM con pesos distintos, estabilidad de Dijkstra ante cambios locales, comportamiento de Ford–Fulkerson con capacidades enteras.

Ejercicio 2 – Caminos mínimos: “¿hay más de un camino mínimo?” (20 puntos)

Sea un grafo dirigido ponderado $G = (V, E)$ con pesos positivos en todas las aristas ($w(e) > 0$). Se fija un vértice origen s y un vértice destino t .

Queremos decidir si existe más de un camino mínimo (en costo total) desde s hasta t ; es decir, si existen al menos dos caminos diferentes $P1 \neq P2$ tales que:

$$\text{costo}(P1) = \text{costo}(P2) = \text{dist}(s, t).$$

1) (10 pts)

Diseñe un algoritmo eficiente que, dado G , s y t , decida si existe más de un camino mínimo s - t .

- Puede usar una sola corrida de Dijkstra desde s como subrutina.

- Describa claramente:

* Qué información obtiene con Dijkstra.

* Qué subgrafo construye a partir de esa información.

* Cómo decide, sobre ese subgrafo, si hay más de un camino mínimo.

- Analice la complejidad temporal total de su algoritmo en función de $|V|=n$ y $|E|=m$.

2) (10 pts)

Justifique por qué su algoritmo es correcto. En particular:

a) Explique por qué, si $\text{dist}(s, \cdot)$ son las distancias calculadas por Dijkstra, una arista (u, v) puede pertenecer a algún camino mínimo de s a t solo si cumple cierta relación entre $\text{dist}(s, u)$, $w(u, v)$ y $\text{dist}(s, v)$ (indique claramente cuál).

b) Argumente por qué el subgrafo que construye contiene exactamente las aristas que pueden aparecer en algún camino mínimo s - t .

c) Explique por qué, en ese subgrafo, el problema de “¿existe más de un camino mínimo?” se reduce a

“¿existen al menos dos caminos dirigidos distintos de s a t ?”, y por qué la técnica que usted propone

(por ejemplo, un conteo de caminos con DP sobre un DAG, o alguna otra) es correcta.

Tema que evalúa: uso fino de Dijkstra, caracterización de aristas que pertenecen a caminos mínimos, construcción de subgrafos “ s - t -eficientes”, uso de DP en DAG o ideas similares para contar caminos, corrección y complejidad.

Ejercicio 3 – Propiedades del AGM y verificación de optimalidad (20 puntos)

Sea un grafo no dirigido, conexo, con pesos positivos en las aristas.

1) (8 pts)

Demuestre la siguiente afirmación (puede usar propiedades de corte/ciclo vistas en clase):

“En un grafo no dirigido y ponderado con pesos todos distintos, para cualquier ciclo simple C , la arista de mayor peso de C no pertenece a ningún árbol generador mínimo del grafo.”

La demostración puede ser informal pero debe ser lógica y clara (no alcanza con “es lo que dice la teórica”).

2) (8 pts)

Usando el resultado del inciso anterior, diseñe un algoritmo que, dado:

- Un grafo no dirigido conexo $G = (V, E)$ con pesos todos distintos,
 - y un árbol generador T de G ,
- determine si T es efectivamente un árbol generador mínimo de G .

- Describa el algoritmo a alto nivel (puede suponer que dispone de funciones auxiliares razonables).

- Analice su complejidad temporal en función de $n = |V|$ y $m = |E|$.

- No es necesario que el algoritmo sea el más óptimo posible, pero sí que esté correctamente justificado.

3) (4 pts)

Discuta brevemente qué cambia si se permiten pesos repetidos (es decir, no todos los pesos son distintos).

- ¿Sigue siendo cierto que la arista de mayor peso en cualquier ciclo no pertenece a ningún AGM?

- ¿Qué impacto tiene esto en su algoritmo de verificación? ¿Sigue funcionando tal cual, hay que ajustarlo, deja de servir?

Tema que evalúa: propiedades de corte/ciclo del AGM, relación entre ciclos y aristas “pesadas”, diseño de algoritmos de verificación de optimalidad, impacto de la unicidad del AGM vs existencia de múltiples AGM.

Ejercicio 4 – Flujo con capacidad en vértices (30 puntos)

Sea un digrafo $D = (V, E)$ con un origen $s \in V$ y un sumidero $t \in V$.

Cada arista $(u, v) \in E$ tiene una capacidad $c(u, v) \in \mathbb{Z}_{\geq 0}$ como en una red de flujo clásica.

Además, cada vértice $v \in V \setminus \{s, t\}$ tiene una capacidad de vértice $b(v) \in \mathbb{Z}_{\geq 0}$, que representa la cantidad máxima de flujo

total que puede “pasar por” ese vértice (sumando todo lo que entra/sale).

Queremos enviar flujo desde s hasta t respetando tanto las capacidades de aristas como las capacidades de vértices.

1) (12 pts)

Modele este problema como un problema estándar de flujo máximo (solo con capacidades en aristas). Es decir, construya una red $G' = (V', E')$

y capacidades c' tales que:

- Cualquier flujo factible en D (respetando capacidades de aristas y de vértices) corresponde a un flujo factible en G' de igual valor.
- Y viceversa: cualquier flujo factible en G' corresponde a un flujo factible en D .

Describa claramente:

- Cómo transforma cada vértice v (incluya qué hace con s y t).
- Qué aristas agrega y con qué capacidades.
- Cómo se mapea un flujo de D a G' y al revés (al menos a nivel conceptual).

2) (10 pts)

Explique por qué su modelo es correcto. En particular:

- Justifique por qué la capacidad de vértice $b(v)$ se respeta en G' usando solo capacidades de aristas.
- Argumente por qué no está “creando” flujos que no son posibles en D (por ejemplo, agregando caminos raros que no existen en el grafo original).
- Mencione brevemente el rol de la conservación de flujo en los vértices divididos.

3) (8 pts)

Suponga que en D se cumplen:

- $n = |V|$, $m = |E|$.
- Todas las capacidades de aristas y vértices son enteras y están acotadas por una constante U .

a) Exprese (en términos de n y m) cuántos vértices y cuántas aristas puede llegar a tener a lo sumo la red G' que usted construyó.

b) Si corre Edmonds–Karp sobre G' , dé una cota asintótica de la complejidad temporal total en función de n y m

(no hace falta que sea la más apretada posible, pero sí razonada).

c) Comente brevemente si el uso de capacidades en vértices cambia de alguna forma la integridad de las soluciones

(es decir, si sigue siendo cierto que existe un flujo máximo entero cuando todas las capacidades son enteras).

Tema que evalúa: modelado con flujo (vértices con capacidad $\rightarrow$ splitting), construcción de red equivalente, corrección del modelo, análisis de complejidad después de la transformación, integridad de soluciones de flujo.

Ejercicio 5 – Preguntas cortas integradoras (20 puntos)

Responder en 3–6 líneas por inciso. Cada ítem vale 5 puntos.

1) (5 pts)

Explique en qué condiciones es correcto usar BFS para calcular caminos mínimos desde un origen s en un grafo.

- ¿Qué restricciones deben cumplir los pesos de las aristas?

- ¿Qué complejidad temporal tiene BFS en términos de n y m , usando listas de adyacencia?

2) (5 pts)

Sea un grafo dirigido ponderado que puede tener aristas de peso negativo, pero no tiene ciclos de peso negativo alcanzables desde s .

- Explique brevemente por qué Dijkstra puede devolver distancias incorrectas en este contexto, incluso sin ciclos negativos.

- ¿Qué algoritmo usaría en su lugar para caminos mínimos desde un solo origen? Indique también su complejidad temporal.

3) (5 pts)

Describa brevemente el algoritmo de Bellman–Ford:

- Qué problema resuelve (qué tipo de grafos admite en cuanto a pesos de aristas).

- Cómo puede utilizarse específicamente para detectar la existencia de un ciclo de peso negativo alcanzable desde el origen.

4) (5 pts)

Compare el uso de Floyd–Warshall con el enfoque de correr Dijkstra desde varios orígenes para calcular caminos mínimos entre muchos pares de vértices.

- Mencione la complejidad temporal de Floyd–Warshall en función de n .

- Comente en qué situaciones conviene usar Floyd–Warshall (por ejemplo, cuando se quieren distancias para casi todos los pares)

y en cuáles conviene usar “varios Dijkstra” (por ejemplo, cuando solo interesan pocos orígenes o el grafo es muy raro).